

P2085R0: Consistent defaulted comparisons

Audience: EWG, CWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

February 14, 2020

Background

Special member functions may be defaulted in or out of their class:

```
class MysteryCopy {
    int a, b, c, d, e, f, g;
public:
    MysteryCopy()=default;           // value initialization zeros
    MysteryCopy(const MysteryCopy&); // not trivially copyable
};

MysteryCopy::MysteryCopy(const MysteryCopy&)=default;
// still not trivially copyable; equivalent to
// MysteryCopy::MysteryCopy(const MysteryCopy &m) /* noexcept(false) */
// : a(m.a), b(m.b), c(m.c), d(m.d), e(m.e), f(m.f), g(m.g) {}
```

The separate definition allows the programmer to control special properties of the function (`constexpr` and `inline`) and of the class (various kinds of triviality) without having to write a long definition manually (here, to copy each member). In particular, unless the out-of-class definition is declared `inline`, it will typically appear in a separate translation unit and will provide ABI stability (as described in [dcl.fct.def.default]/5) and avoid duplicate code generation. It's even possible to declare the function `inline`, `constexpr`, or `constexpr` in the class but provide its defaulted definition (and thus allow its use) only in certain translation units. (By contrast, the exception specification is controllable even in the class via `noexcept(false)`.)

The situation for comparison functions is less clear. [P0515R3](#) merely requires that a defaulted comparison be “declared in the *member-specification* of C”, which can be taken to allow `=default` on a redeclaration of a friend. In fact, that paper goes further and mentions the possibility of access checks for non-member, *non-friend* defaulted operator`<=>`. [P0732R2](#) defines strong structural equality in terms of an operator`<=>` defaulted in the definition of a class, as would be compatible with being able to do so elsewhere. [P1185R2](#) preserves these wording details when separating `==` from `<=>` and does not mention the possibility at all.

There is no record of any Evolution discussion of where to allow `=default` since [N3950](#), whose revisions [N4114](#) and [N4126](#) included “unanimously requested” support for non-members. [P0432R0](#), which revived the syntax, retained that support.

However, the recollection of Core (at the January 16 issues-processing teleconference) is that the intent was to require that `=default` be used only within a class for comparisons, and [P2002R1](#) clarifies the rule to say so. Although [P1907R1](#) removed the relevant class property of strong structural equality, this restriction does have the effect of making it *impossible* to default a comparison without making it `inline` (and possibly `constexpr`). This seems undesirable from an expressivity standpoint; one of the reasons for introducing default comparisons was to avoid writing error-prone repetitive code manually, but anyone who wants to control the properties of a comparison operator must do so (and potentially reimplement the synthesized three-way comparisons from [class.spaceship]/1). It also seems needlessly inconsistent with the other functions (the special members) that can be defaulted.

There is another reason for comparison functions to not be inline: the linkage concerns in [P1498R1](#). Those issues matter little for defaulted special member functions, which principally call member functions of types that already must be available to any translation unit that can call the defaulted function. For that reason and for consistency reasons, [P1779R3](#) does not affect the implicit inline for functions defaulted in a class. However, it would not be at all unexpected for a defaulted comparison to invoke non-member subobject comparisons with internal linkage (because they are meaningless to clients). Such a comparison would be allowed by [P1815R1](#) only if it were made opaque by being defined outside a class.

Proposal

To allow defaulted comparisons to be non-inline (to control ABI or code generation), or inline but not constexpr (even if constexpr-compatible), allow their definition to appear outside the class that declares them (possibly as a friend). Retain the restriction that they be *declared* in the relevant class to avoid encouraging clients to add to a class's interface (and to avoid questions of access).

Hidden friend comparison operators can be defaulted in an implementation translation unit, or in a *private-module-fragment*, to avoid being inline while remaining hidden. A member of, or an unbounded set of friends injected by, a class template cannot be so isolated, but clients must be able to generate code for them anyway.

Tony table

N4849

```
struct Ints {
    int i,j,k;
    // since we had to add k recently:
    // std::unique_ptr<int[]> rest;

    // This can't be constexpr once we add
    // 'rest', so don't rely on that now.
    friend bool operator==(const Ints&,const
                           Ints&);
};
inline bool operator==(const Ints &a,const
                       Ints &b) {
    return a.i==b.i && a.j==b.j; // oops
}
```

this proposal

```
struct Ints {
    int i,j,k;
    // since we had to add k recently:
    // std::unique_ptr<int[]> rest;

    // This can't be constexpr once we
    // add 'rest', so don't rely on
    // that now.
    friend bool operator==(const
                           Ints&,const Ints&);
};
inline bool operator==(const
                       Ints&,const Ints&)=default;
```

weak.hpp:

```
#ifndef WEAK_HPP
#define WEAK_HPP

#include<compare>

struct Legacy {
    // ...
    // These define a weak order:
    bool operator==(const Legacy&) const { /*
        lots of code */
    }
    bool operator<(const Legacy&) const { /* lots
        of code */
    }
};
```

weak.hpp:

```
#ifndef WEAK_HPP
#define WEAK_HPP

#include<compare>

struct Legacy {
    // ...
    // These define a weak order:
    bool operator==(const Legacy&) const
        { /* lots of code */
    }
    bool operator<(const Legacy&) const
        { /* lots of code */
    }
};
```

```

struct Spaceship {
    Legacy a,b,c;
    // This is a lot of code; let's generate it once:
    std::weak_ordering operator<=>(const
        Spaceship&) const;
};

```

```

#endif

```

weak.cpp:

```

#include "weak.hpp"

```

```

std::weak_ordering Spaceship::operator<=>
    (const Spaceship &s) const {
    if (!(a==s.a))
        return a<s.a ? std::weak_ordering::less :
            std::weak_ordering::greater;
    if (!(b==s.b))
        return b<s.b ? std::weak_ordering::less :
            std::weak_ordering::greater;
    if (!(c==s.c))
        return c<s.c ? std::weak_ordering::less :
            std::weak_ordering::greater;
    return std::weak_ordering::equivalent;
}

```

```

export module A;
import<compare>;

class Helper {
    // ...
};
namespace { // users shouldn't call this; it's surprisingly expensive
    std::partial_ordering operator<=>(const
        Helper &a, const Helper &b) {
        // ...
    }
}

export class Public {
    int i;
    Helper h1,h2;
public:
    // ...
    std::partial_ordering operator<=>(const
        Public&) const;
};
std::partial_ordering Public::operator<=>
    (const Public &p) const {
    // can't use return type deduction: this would be std::strong_ordering
    if(const auto v=i<=>p.i; v!=0) return v;
    if(const auto v=h1<=>p.h1; v!=0) return v;
    return h2<=>p.h2;
}

```

```

struct Spaceship {
    Legacy a,b,c;
    // This is a lot of code; let's generate it once:
    std::weak_ordering operator<=>(const
        Spaceship&) const;
};

```

```

#endif

```

weak.cpp:

```

#include "weak.hpp"

```

```

std::weak_ordering
    Spaceship::operator<=>(const
        Spaceship&) const=default;

```

```

export module A;
import<compare>;

class Helper {
    // ...
};
namespace { // users shouldn't call this; it's surprisingly expensive
    std::partial_ordering operator<=>
        (const Helper &a, const Helper &b) {
        // ...
    }
}

export class Public {
    int i;
    Helper h1,h2;
public:
    // ...
    auto operator<=>(const Public&)
        const;
};
auto Public::operator<=>(const Public&)
    const=default;

```

Wording

Relative to N4849

[class.compare.default]

Change paragraph 1:

- A defaulted comparison operator function ([over.binary]) for some class C shall be a non-template function ~~declared in the member-specification of C~~ that is
 1. a non-static const member of C having one parameter of type const C&, or
 2. a friend of C having either two parameters of type const C& or two parameters of type C.

Change paragraph 3:

- If the class definition does not explicitly declare an == operator function, but declares a ~~defaulted~~ three-way comparison operator function as defaulted, an == operator function is declared implicitly with the same access as the three-way comparison operator function. The implicitly-declared == operator for a class X is an inline member and is defined as defaulted in the definition of X. If the three-way comparison operator function is declared as a non-static const member, the implicitly-declared == operator function is a member of the form

[...]

Relative to N4849 as modified by P2002R1

[class.compare.default]

Change paragraph 1:

- A defaulted comparison operator function ([over.binary]) for some class C shall be a non-template function ~~defined in the member-specification of C~~ that is
 1. a non-static const non-volatile member of C having one parameter of type const C& and either no *ref-qualifier* or the *ref-qualifier* &, or
 2. a friend of C having either two parameters of type const C& or two parameters of type C.

A ~~defaulted~~ comparison operator function for class C that is defaulted on its first declaration and is not defined as deleted is *implicitly defined* when it is odr-used or needed for constant evaluation. Name lookups in the defaulted definition of a comparison operator function are performed from a context equivalent to the its function-body of the defaulted operator@function. ~~A defaulted~~ A definition of a comparison operator function shall be as defaulted on its that appears in a class shall be the first declaration of that function.

Change paragraph 3 (editorially, for clarity):

- If the *member-specification* does not explicitly declare any member or friend named operator==, an == operator function is declared implicitly for each ~~defaulted~~ three-way comparison operator function defined as defaulted in the *member-specification*, with the same access and *function-definition* and in the same class scope as the three-way comparison operator function, except that the

return type is replaced with `bool` and the *declarator-id* is replaced with `operator==`. [Note: [...] — *end note*] [Example:

[...]

— *end example*] [Note: [...] — *end note*]

Acknowledgments

Thanks to Richard Smith for helping identify use cases.